

WEEK 3: SAREE BOUTIQUE – SELLER AND INVENTORY MANAGEMENT SYSTEM

SCENARIO

The Saree Boutique sells different types of sarees supplied by multiple sellers. The system should maintain seller details, track sarees supplied by each seller, and monitor inventory levels to ensure saree availability.

1. CREATE SELLER AND INVENTORY TABLES

Create Seller and Inventory Tables for the Saree Boutique.

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone NUMBER(10), Address VARCHAR(45));
Table created.

SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7), Product_ID VARCHAR(7), Seller_ID VARCHAR(7), Stock_Quantity NUMBER, Stock_status VARCHAR(11), Last_Updated DATE);
Table created.
```

2. ESTABLISH RELATIONSHIPS

Seller and Saree Product

One seller can supply multiple saree products.

One saree product can be supplied by one or multiple sellers.

Relationship:

Seller → Product is one to many

Product and Inventory

Each saree product has inventory information.

Inventory tracks available saree stock.

Relationship:

Product → Inventory is one to one

PRIMARY KEY ADDED FOR BOTH TABLES

Primary Key is added to uniquely identify each record in the tables.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);
Table altered.
```

FOREIGN KEY FOR INVENTORY TABLE

Foreign Key is added to establish the relationship between the Inventory table and Product table.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);
Table altered.
```

CONSTRAINTS TO MAKE UNIQUE EMAIL ID

A Unique constraint is added to the Email column of the Seller table.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);
Table altered.
```

TO MAKE EMAIL NOT NULL

The Email column is made NOT NULL so that every seller must have an email ID.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;
Table altered.
```

ADD CONSTRAINT TO CHECK THE STOCK QUANTITY IS POSITIVE

A CHECK constraint is added to ensure that the Stock Quantity is positive.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity >=0);
Table altered.
```

INSERT VALUES TO SELLER TABLE

Insert sample seller records into the Seller table for testing and report generation.

```
SQL> INSERT INTO SELLER VALUES('&SellID','&SellerName','&email','&phone','&Address');
Enter value for sellid: 1
Enter value for sellername: Yamini
Enter value for email: yamini@gmail.com
Enter value for phone: 7845220473
Enter value for address: 12, Anna Nagar 3rd St, Chennai
old 1: INSERT INTO SELLER VALUES('&SellID','&SellerName','&email','&phone','&Address')
new 1: INSERT INTO SELLER VALUES('1','Yamini ','yamini@gmail.com','7845220473','12, Anna Nagar 3rd St, Chennai')

1 row created.

SQL> /
Enter value for sellid: 2
Enter value for sellername: NANDY
Enter value for email: nandy@gmail.com
Enter value for phone: 9445961241
Enter value for address: 24, Gandhi Rd 5th St, Tanbaram
old 1: INSERT INTO SELLER VALUES('&SellID','&SellerName','&email','&phone','&Address')
new 1: INSERT INTO SELLER VALUES('2','NANDY','nandy@gmail.com','9445961241','24, Gandhi Rd 5th St, Tanbaram')

1 row created.
```

```
SQL> SELECT * FROM SELLER;
```

SELLER_ID	SELLER_NAME	EMAIL	PHONE	LOCATION
1	Yamini	yamini@gmail.com	7845220473	12, Anna Nagar 3rd St, Chennai
2	NANDY	nandy@gmail.com	9445961241	24, Gandhi Rd 5th St, Tambaram

```

SELLER_ID SELLER_NAME
-----
EMAIL
-----
LOCATION
-----

3 HEMA
hema@gmail.com
Lakshmi Nagar, Chengalpattu

4 abi
abi@gmail.com

SELLER_ID SELLER_NAME
-----
EMAIL
-----
PHONE
-----
LOCATION
-----

Kamarajar St, Madipakkam\

5 deeps
deeps@gmail.com
jeeva nagar ,parari

```

INSERT VALUES TO INVENTORY TABLE

Saree products used:

```
SQL> INSERT INTO Inventory VALUES('&InvID','&ProdID','&Stock','&Stock_status');
Enter value for invId: 1
Enter value for prodId: P001
Enter value for stock: 10
Enter value for stock_status: Available
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Stock','&Stock_status')
new 1: INSERT INTO Inventory VALUES('1','P001','10','Available')
```

```
SQL> SELECT * FROM INVENTORY;
```

INVENTORY_ID	PRODUCT	STOCK_QUANTITY	STOCK_STATUS
1	P001	10	Available
2	P002	15	Available
4	P002	8	Available
3	P001	20	Available

REPORT 1: SELLER-WISE PRODUCT REPORT

Display:

- Seller name
- Number of saree products supplied
- Available saree stock

```
SQL> SELECT S.SELLER_NAME, COUNT(I.PRODUCT_ID) AS NUMBER_OF_PRODUCTS, SUM(I.STOCK_QUANTITY) AS AVAILABLE_STOCK FROM SELLER S JOIN INVENTORY I ON S.SELLER_ID=I.SELLER_ID WHERE I.STOCK_STATUS='Available' GROUP BY S.SELLER_NAME;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
HEMA	1	20
Yamini	1	10
abi	1	8
NANDY	1	15

REPORT 2: STOCK AVAILABILITY REPORT

Display:

- Saree product name
- Stock quantity
- Stock status

```
SQL> SELECT P.PRODUCT_NAME, I.STOCK_QUANTITY, I.STOCK_STATUS FROM PRODUCT1 P JOIN INVENTORY I ON P.PRODUCT_ID=I.PRODUCT_ID;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Kanchipuram Silk Saree	10	Available
Cotton Saree	15	Available
Cotton Saree	8	Available

REPORT 3: INVENTORY ANALYSIS

Find:

- Total saree products available
- Saree products out of stock
- Highest stocked saree products
- Average inventory quantity

```
SQL> SELECT COUNT(*) AS TOTAL_PRODUCTS_AVAILABLE, SUM(CASE WHEN STOCK_QUANTITY=0 THEN 1 ELSE 0 END) AS PRODUCTS_OUT_OF_STOCK, MAX(STOCK_QUANTITY) AS HIGHEST_STOCK, AVG(STOCK_QUANTITY) AS AVERAGE_INVENTORY_QUANTITY FROM INVENTORY;
```

TOTAL_PRODUCTS_AVAILABLE	PRODUCTS_OUT_OF_STOCK	HIGHEST_STOCK	AVERAGE_INVENTORY_QUANTITY
4	0	20	13.25

HIGHEST STOCKED SAREE PRODUCTS

```
SQL> SELECT P.PRODUCT_NAME, I.STOCK_QUANTITY FROM PRODUCT1 P JOIN INVENTORY I ON P.PRODUCT_ID=I.PRODUCT_ID WHERE I.STOCK_QUANTITY=(SELECT MAX(STOCK_QUANTITY) FROM INVENTORY);
```

PRODUCT_NAME	STOCK_QUANTITY
Kanchipuram Silk Saree	20

AVERAGE INVENTORY QUANTITY

SELECT AVG(STOCK_QUANTITY) AS AVERAGE_INVENTORY_QUANTITY FROM INVENTORY;

```
SQL> SELECT AVG(STOCK_QUANTITY) AS AVERAGE_INVENTORY_QUANTITY FROM INVENTORY;
```

AVERAGE_INVENTORY_QUANTITY
13.25